and ## Operators in C

Last Updated : 02 Jan, 2025

In C, **# and ## operators** are preprocessor operators using in macros for token manipulation. They are known as **stringizing** and **token pasting** operators and are used in macro definition with #define preprocessor. In this article, we will learn about these operators and how to use them in C programs.

Stringizing Operator (#)

The **stringizing operator (#)** is a preprocessor operator that causes the corresponding actual argument to be enclosed in **double quotation marks.** When the # operator is placed before an argument in a macro definition, it converts that argument into a string literal. It is generally used with macros in C.

Syntax

#define MACRO_NAME(arg) #arg

Example

```
#include <stdio.h>

// Macro definition using stringification (#)
// convert the argument x into a string
#define PRINT_STRING(x) printf(#x)

int main() {

    // Using the macro
    PRINT_STRING(Hello Geeks);

    return 0;
}
```

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy

Got It !

```
Hello Geeks
```

Explanation: The **# operator** in the macro **PRINT_STRING(x)** converts the argument **x** into a string by enclosing it into double quotes. The statement **PRINT_STRING(Hello Geeks)** is hence converted into **printf("Hello Geeks")**.

> ***Note:*** *This operator should not be confused with the # operator used for defining preprocessor directive.*

Token Pasting Operator (##)

The t**oken-pasting operator (##)** allows tokens used as actual arguments to be concatenated to form other tokens. It is often useful to merge two tokens into one while expanding macros. This is called token pasting or token concatenation.

The ## pre-processing operator performs token pasting. When a macro is expanded, the two tokens on either side of each '##' operator are combined into a single token, which then replaces the '##' and the two original tokens in the macro expansion.

Syntax

> *#define MACRO_NAME(arg1, arg2) arg1 ## arg2*

Examples:

```c
#include <stdio.h>

// Macro definition using the Token-pasting operator
#define concat(a, b) a##b
int main(void) {
    int xy = 30;

    // Printing the concatenated value of x and y
    printf("%d", concat(x, y));
    return 0;
}
```

Output

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy

Explanation: The ## operator in the macro concat(a, b) is the token-pasting operator. It concatenates two tokens (a and b) into a single token. In this example, concat(x, y) concatenates x and y to form xy, which refers to the variable int xy = 30;. Therefore, printf("%d", concat(x, y)); prints the value of xy, which is 30.

Application of Token-pasting operator (##)

The ## provides a way to concatenate actual arguments during macro expansion. If a parameter in the replacement text is adjacent to a ##, the parameter is replaced by the actual argument, the ## and surrounding white space are removed, and the result is re-scanned.

Predict the output of the following program:

```
#include <stdio.h>

#define combine(a, b, c, d, e, f, g) c ## d ## e ## f
#define start combine(m, a, i, n, p, r, o)

int start()
{
    printf("Did you figure it out?\n");
    return 0;
}
```

Comment | More info | Advertise with us

Next Article

How to print a variable name in C?

Similar Reads

C Preprocessors

Preprocessors are programs that process the source code before the actual compilation begins. They are not part of the compilation process but operate separately, allowing programmers to modify the code...

8 min read

C Preprocessor Directives

In C programming, the preprocessor is a program that process the source code before the actual

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy

Compiling a C program - Behind the Scene A Preprocessor is a system software (a computer program that is designed to run on computer's hardware and application programs). It performs preprocessing of the...

3 min read

Header Files in C

In C programming, a header file is a file that ends with the .h extension and contains features like functions, data types, macros, etc that can be used by any other C program by including that particular...

5 min read

Whatâ€™s difference between header files "stdio.h" and "stdlib.h" ?

In C programming, standard header files provide various inbuilt functionalities and two of the most commonly used standard header files are stdio.h and stdlib.h. The <stdio.h> provides Standard Input...

4 min read

How to write your own header file in C?

As we all know that files with .h extension are called header files in C. These header files generally contain function declarations which we can be used in our main C program, like for e.g. there is need to...

4 min read

Macros and its types in C

In C programming, a macro is a symbolic name or constant that represents a value, expression, or code snippet. They are defined using the #define directive, and when encountered, the preprocessor substitute...

4 min read

Interesting Facts about Macros and Preprocessors in C

In a C program, all lines that start with # are processed by preprocessor which is a special program invoked by the compiler. by this we mean to say that the â€˜#â€™ symbol is used to process the...

5 min read

and ## Operators in C

In C, # and ## operators are preprocessor operators using in macros for token manipulation. They are known as stringizing and token pasting operators and are used in macro definition with #define...

3 min read

How to print a variable name in C?

Printing a variable name means printing the identifier that is assigned to the variable. To print it, it should be in the form of string. This can be done by using stringification.Stringification in C is the method to...

1 min read

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy

A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

GET IT ON Google Play
Download on the App Store

Advertise with us

Company
About Us
Legal
Privacy Policy
In Media
Contact Us
Advertise with us
GFG Corporate Solution
Placement Training Program

Languages
Python
Java
C++
PHP
GoLang
SQL
R Language
Android Tutorial
Tutorials Archive

DSA
Data Structures
Algorithms
DSA for Beginners
Basic DSA Problems
DSA Roadmap
Top 100 DSA Interview Problems
DSA Roadmap by Sandeep Jain
All Cheat Sheets

Data Science & ML
Data Science With Python
Data Science For Beginner
Machine Learning
ML Maths
Data Visualisation
Pandas
NumPy
NLP
Deep Learning

Web Technologies
HTML
CSS
JavaScript
TypeScript
ReactJS
NextJS
Bootstrap
Web Design

Python Tutorial
Python Programming Examples
Python Projects
Python Tkinter
Python Web Scraping
OpenCV Tutorial
Python Interview Question
Django

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy

Database Management System
Software Engineering
Digital Logic Design
Engineering Maths
Software Development
Software Testing

AWS
Docker
Kubernetes
Azure
GCP
DevOps Roadmap

System Design

High Level Design
Low Level Design
UML Diagrams
Interview Guide
Design Patterns
OOAD
System Design Bootcamp
Interview Questions

Inteview Preparation

Competitive Programming
Top DS or Algo for CP
Company-Wise Recruitment Process
Company-Wise Preparation
Aptitude Preparation
Puzzles

School Subjects

Mathematics
Physics
Chemistry
Biology
Social Science
English Grammar
Commerce
World GK

GeeksforGeeks Videos

DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy